configured to reexecute the pipeline, it generates a Problem Details response by using the `IProblemDetailsService`.

Listing 5.6 shows how to configure Problem Details generation in your exception handlers. Add the required `IProblemDetailsService` service to your app, and call `UseExceptionHandler()` without providing an error-handling path, and the middleware will generate a Problem Details response automatically when it catches an exception.

Listing 5.6 Configuring `ExceptionHandlerMiddleware` to use Problem Details

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
builder.Services.AddProblemDetails();                ❶

WebApplication app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler();                       ❷
}

app.MapGet("/", void () => throw new Exception());   ❸

app.Run();
```

❶ Adds the IProblemDetailsService implementation

❷ Configures the ExceptionHandlerMiddleware without a path so that it uses the IProblemDetailsService

❸ Throws an exception to demonstrate the behavior

As discussed in chapter 4, `WebApplication` automatically adds the `DeveloperExceptionPageMiddleware` to your app in the development environment. This middleware similarly supports returning Problem Details when two conditions are satisfied: